

Quasi-Steady-State Racing Line and Lap Time Optimization Using Discrete Point-Force Dynamics

Barnabas Szeibert (5474450), Panagiotis Sachinis (5526957)

April 2026

1 Introduction and Motivation

In a field as competitive as the motorsport industry, extracting the maximum performance from a vehicle is as, or even more, critical than designing the vehicle itself. In a given (race) session, the ultimate target of a team and driver is to minimize the time in which a single or a series of laps is completed (referred to as the lap-time). The lap time a vehicle with a given performance can achieve is limited by a myriad of variables. One of these, and perhaps the easiest one to adjust 'on the fly' is the path the driver chooses to navigate through a circuit, with the goal of maximizing the average velocity, and as such, minimizing the lap time. This optimal path is often referred to as the racing line, and is a core principle in dynamic motorsport optimization.

Determining an optimal racing line is an extremely complex and highly-dimensional problem, as the ideal path varies on a multitude of factors. Firstly, these include physical considerations of a given racetrack, such as track width, corner radius, and track surface conditions, as well as higher order factors such as sector dependency (the velocity and position on the exit of one corner directly affects the entry into the next). These geometric considerations are then directly coupled to the dynamic performance of the vehicle, including longitudinal power and braking limitations, aerodynamic and tire friction forces, and many more factors. As a result, characteristics of an optimal racing line will vary greatly between different race tracks and different vehicles (and by extension, vehicle configurations).

Despite this substantial sensitivity to various physical factors, decades of motorsport research have shown that an optimal path does indeed exist for a given circuit. As the goal is to maintain the highest average velocity over a minimum distance, this problem does not reduce to maximizing velocity or minimizing distance. Instead, it may be decoupled into two separate problems. Firstly, to reduce physical losses, the general concept of a racing line relies on minimizing the sum of the curvature over the entire lap of a circuit. By taking into consideration the physical performance restrictions of the vehicle, a geometrically optimized line may be adjusted to account for higher order effects and create a 'drive-able' optimized path.

As such, this paper will aim to determine the optimal racing line for a given circuit by first reducing the order of the problem into a purely geometric optimization, and then

using multiple methods to determine an optimal physical path. Firstly, the numerical and physical methodology is defined and discussed in Section 2. Then, the optimization problem definitions, methodologies, and algorithm choices will be discussed in Section 3. Finally, the results and discussion for the two optimizations will be discussed in Section 4.

2 Modeling Methodology

Before a numerical solver can determine a time-optimal trajectory, the physical realities of the vehicle and the racing circuit must be translated into a mathematical framework. This section presents all the methods used to build the mathematical model, such as track definition, discretization and node interpolation.

2.1 Track Representation

Cartesian coordinates of various circuits were obtained as 2D projections of the circuit dimensions, and may be found here [1]. To mathematically describe the trajectory of a vehicle around a (2D projected) circuit, a track-relative curvilinear coordinate system is established. Rather than optimizing the vehicle’s path purely in a global Cartesian coordinate system, the spatial domain is mapped relative to the geometric centerline of the track.

Let the track centerline be parameterized by its longitudinal arc length s , where $s \in [0, S]$, and S is the total geometric length of the closed circuit. At any given distance s , the centerline is defined by a two-dimensional Cartesian position vector $\mathbf{r}_c(s) = [X_c(s), Y_c(s)]^T$.

To establish the lateral domain and the physical boundaries of the drivable surface, an orthogonal unit normal vector $\mathbf{n}(s)$ is defined at every point along the centerline. By convention for this model, $\mathbf{n}(s)$ points towards the left boundary of the track. The track limits are then defined by two distance functions, $w_{left}(s)$ and $w_{right}(s)$, which represent the orthogonal distances from the centerline to the left and right track edges, respectively.

The path of the vehicle, typically referred to as the racing line, is defined by a single spatially-dependent design variable, $\alpha(s)$. This variable represents the lateral deviation of the vehicle’s center of gravity from the track centerline at longitudinal distance s . Given that the normal vector points left, a positive α denotes a position on the left side of the track, while a negative α denotes a position on the right. Consequently, the global Cartesian coordinates of the vehicle’s racing trajectory, $\mathbf{r}_{race}(s)$, can be completely expressed as a function of the static centerline geometry and the dynamic lateral deviation:

$$\mathbf{r}_{race}(s) = \mathbf{r}_c(s) + \alpha(s)\mathbf{n}(s) \quad (1)$$

By formulating the spatial domain in this manner, the complex two-dimensional routing problem is effectively reduced to a one-dimensional optimization problem. The numerical solver is no longer required to guess (X, Y) coordinates blindly, it solely needs to determine the optimal lateral deviation α at any given longitudinal progression s .

2.2 Adaptive Node Placement

Tracks vary significantly in their topological characteristics, encompassing a diverse array of straightaways and complex corner profiles. In motorsport trajectory optimization, straight line segments require minimal spatial resolution, as the optimal path is mathematically trivial. Consequently, a linear, equal distribution of control nodes across the entire track length is computationally inefficient. Conversely, corners demand substantially higher resolution. To overcome the inefficiencies of linear node spacing, an Adaptive Mesh Refinement (AMR) technique is implemented. This approach utilizes the track's topological curvature to generate a density function, which dynamically pulls control points away from the straights and clusters them densely around the critical braking zones and apexes. The process begins by calculating the raw curvature, κ , of the track's geometric centerline at any given distance s . Let the centerline be defined by the physical coordinates $(X(s), Y(s))$. The raw curvature is given by:

$$\kappa(s) = \frac{\dot{X}(s)\ddot{Y}(s) - \dot{Y}(s)\ddot{X}(s)}{(\dot{X}(s)^2 + \dot{Y}(s)^2)^{3/2}} \quad (2)$$

As raw track data contains high-frequency spatial noise, relying on $\kappa(s)$ directly can result in numerical instability or the identification of false micro-apexes. To resolve this, a Gaussian smoothing window is applied to the absolute curvature, yielding a smoothed curvature profile, $\bar{\kappa}(s)$. To ensure the algorithm performs consistently across varying circuits, regardless of whether the track consists of tight hairpins or fast sweepers, the smoothed curvature is normalized against the maximum curvature found on the track:

$$\hat{\kappa}(s) = \frac{\bar{\kappa}(s)}{\max(\bar{\kappa})} \quad (3)$$

To further isolate the critical cornering phases, a focusing exponent p is applied (where $p > 1$). This non-linear transformation forces the density to drop off sharply outside of the corner complex, leaving the straights completely sparse:

$$\kappa_f(s) = \hat{\kappa}(s)^p \quad (4)$$

A node density function, $\rho(s)$, is then defined. A baseline density of 1 guarantees that long straights still receive a minimum baseline of control points to maintain the spline's structural continuity. The focused curvature is then scaled by a tunable corner weight, w_c , which dictates the gravitational pull of the apexes:

$$\rho(s) = 1 + w_c \cdot \kappa_f(s) \quad (5)$$

To determine the final physical placement of the nodes, the density function is numerically integrated along the full geometric length of the track, L , to produce the cumulative density function, $C(s)$:

$$C(s) = \int_0^s \rho(\tau) d\tau \quad (6)$$

Finally, to generate N adaptive control points, the density domain is sampled at equal intervals from 0 to $C(L)$. These evenly spaced cumulative density samples are then mapped back to the physical track distance s via inverse interpolation. The result is a highly optimized control grid where the physical distance between spatial variables dynamically shrinks as the track curvature increases, providing the numerical solver with maximum mathematical resolution exactly where lap time is most critical.

Figure 1 shows a visualization of the resulting mesh on the Hungaroring with 60 variables.

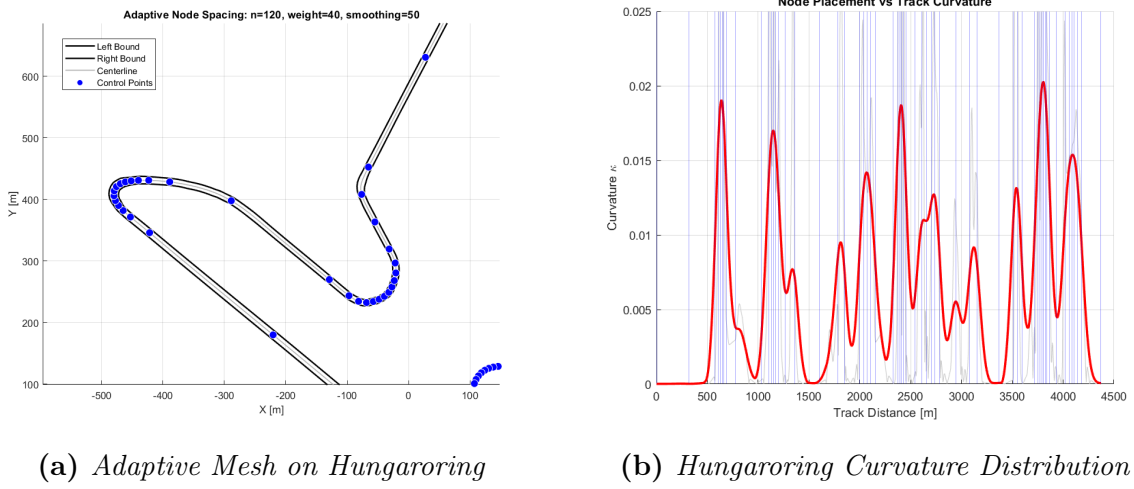


Figure 1: Adaptive Meshing on Hungaroring

2.3 Trajectory Interpolation

While a numerical solver requires a finite set of discrete design variables to function, the ultimate physical objective is to generate a continuous racing line. Building upon the framework established in Section 2.1, these N_{ctrl} generated control points represent the lateral deviations, $\mathbf{u} = [a_{ctrl,1}, a_{ctrl,2}, \dots, a_{ctrl,N_{ctrl}}]^T$, evaluated at their corresponding longitudinal stations, $\mathbf{s}_{ctrl} = [s_1, s_2, \dots, s_{N_{ctrl}}]^T$.

However, to accurately evaluate dynamic physical constraints and kinematic limits across the entire circuit, a continuous trajectory function $a(s)$ must be reconstructed from these discrete nodes. This reconstruction is achieved through piecewise cubic polynomial interpolation. For any given longitudinal interval $s \in [s_i, s_{i+1}]$, the local continuous trajectory is defined by the polynomial equation:

$$P_i(s) = c_{0,i} + c_{1,i}(s - s_i) + c_{2,i}(s - s_i)^2 + c_{3,i}(s - s_i)^3 \quad (7)$$

To ensure geometric continuity, the polynomials must satisfy both position (C^0) and first-derivative (C^1) matching at the nodal boundaries, defined by equations:

$$P_i(s_i) = a_{ctrl,i}, \quad P_i(s_{i+1}) = a_{ctrl,i+1} \quad (8)$$

$$P'_i(s_{i+1}) = P'_{i+1}(s_{i+1}) = d_{i+1} \quad (9)$$

Standard cubic splines solve for the coefficients by rigidly enforcing second-derivative (C^2) continuity across all nodes. However, this global mathematical coupling frequently induces localized spatial overshoot, where the interpolated curve exceeds the local nodal values. In the context of track optimization, this overshoot can force the continuous trajectory to violate physical track limits, even if the discrete control points $a_{ctrl,i}$ and $a_{ctrl,i+1}$ are perfectly legal.

To eliminate this boundary violation, a Modified Akima (makima) piecewise cubic Hermite spline is employed [2]. The MATLAB *makima* algorithm intentionally relaxes the C^2 continuity requirement. Instead, it calculates the nodal derivatives d_i locally based on adjacent differences to mathematically suppress perturbations. Consequently, if the optimizer successfully bounds the discrete control points, the entire continuous piecewise path is guaranteed to remain within the drivable surface:

$$-(W_r(s) - c_m) \leq P_i(s) \leq (W_l(s) - c_m) \quad \forall s \in [0, S] \quad (10)$$

2.4 Physical Model

Accurately modeling the physics of a real race car is an incredibly complex mathematical challenge. In premier motorsport categories, vehicle dynamics encompass thousands of interacting variables. In these highly refined environments, microscopic tweaks to a vehicle's mechanical setup can dictate significant lap time differences.

However, incorporating this level of high-fidelity simulation into a spatial trajectory optimization algorithm is computationally impossible and often introduces unnecessary noise when determining a foundational racing line. Therefore, to formulate a computationally viable optimization problem, the vehicle is reduced to its most fundamental dynamic constraints using the following assumptions:

- **Point-Mass Representation:** The vehicle is idealized as a single point mass, m , concentrated at its center of gravity. This abstracts away the complexities of multidirectional weight transfer (pitch, roll, and yaw) and individual wheel speeds.
- **Constant Friction Coefficient:** The available grip between the tires and the track surface is assumed to be a constant, universal value, represented by the ground friction coefficient, μ . Complex tire behaviors such as thermal degradation and varying slip angles are neglected.
- **Simplified Static Aerodynamics:** While aerodynamic forces are crucial for high-speed cornering, complex phenomena such as fluctuating downforce levels due to ride-height changes or crosswinds are simplified. Aerodynamic forces are modeled strictly as a function of the vehicle's velocity.

2.4.1 Aerodynamic and Normal Forces

The total normal force acting on the car is the sum of the weight and the aerodynamic downforce, F_{DF} . Simultaneously, the vehicle must overcome aerodynamic drag, F_{Drag} . Let ρ be the air density, CLA be the lift coefficient times frontal area, and CDA be the drag coefficient times frontal area. The forces are defined in the following equations:

$$F_{DF} = \frac{1}{2}\rho \cdot \text{CLA} \cdot v^2 \quad (11)$$

$$F_{Drag} = \frac{1}{2}\rho \cdot \text{CDA} \cdot v^2 \quad (12)$$

$$F_z = mg + F_{DF} \quad (13)$$

2.4.2 Lateral Grip Limit

The absolute maximum speed a vehicle can carry through a corner is dictated by the point where the required centripetal force exceeds the maximum available lateral grip (μF_z). The maximum apex speed, v_{limit} , is found by balancing these forces, and the resulting equation is:

$$v_{limit} = \sqrt{\frac{\mu mg}{m|\kappa| - \frac{1}{2}\mu\rho \cdot \text{CLA}}} \quad (14)$$

Note, that taking the absolute value of the curvature (κ) is necessary, as a negative κ simply indicates a right-hand turn relative to the left-pointing normal vector $\mathbf{n}(s)$. The vehicle's lateral grip limit, however, is symmetrically bound by the friction circle and relies strictly on the absolute magnitude of the cornering demand. If the curvature is sufficiently low such that $m|\kappa| \leq \frac{1}{2}\mu\rho \cdot \text{CLA}$, the aerodynamic downforce out scales the cornering demand, meaning the vehicle is entirely drag-limited or engine-limited rather than grip-limited.

2.4.3 Friction Circle

As the tire can only generate a finite amount of total grip, the longitudinal force available for accelerating or braking is constrained by the lateral force currently demanded by the trajectory's curvature. The lateral force is given by $F_{lat} = m|\kappa|v^2$. The remaining available longitudinal force is calculated using the friction circle boundary:

$$F_{lon_avail} = \sqrt{\max(0, (\mu F_z)^2 - F_{lat}^2)} \quad (15)$$

Exceeding this limit would saturate the available tire grip, resulting in brake lockup, tractive wheel spin, or, in the worst-case scenario, a complete loss of vehicle control.

2.4.4 Engine Power and Longitudinal Integration

Forward acceleration is further bounded by the vehicle’s powertrain. The maximum forward thrust is limited by the engine’s peak power output, P_{max} , and the absolute maximum tractive force the lowest gear can transmit, F_{gear_max} :

$$F_{engine} = \min \left(\frac{P_{max}}{v}, F_{gear_max} \right) \quad (16)$$

To determine the final velocity profile $\mathbf{v}$ over the entire closed-loop circuit ($v_1 = v_{N_{pts}}$), a multi-pass integration scheme is utilized. The final velocity at any point i is the geometric minimum of three constraining factors: the absolute lateral grip ceiling, the braking capacity (calculated via backward integration), and the acceleration capacity (calculated via forward integration). This piecewise determination is governed by the following set of equations:

$$v_i^2 = \min \begin{cases} v_{limit,i}^2 \\ v_{i+1}^2 + 2 \left(\frac{F_{lon_avail,i+1} + F_{Drag,i+1}}{m} \right) \Delta s_i \\ v_{i-1}^2 + 2 \left(\frac{\min(F_{lon_avail,i-1}, F_{engine,i-1} - F_{Drag,i-1})}{m} \right) \Delta s_{i-1} \end{cases} \quad (17)$$

Through this multi-pass physical formulation, the optimization algorithm accurately evaluates how track curvature dynamically throttles the vehicle’s speed, allowing the solver to iteratively seek the precise spatial path α that yields the minimum lap time.

2.5 Input Car Parameters

As established in the vehicle dynamics formulation, encapsulating the true dynamic behavior of a race car within a simplified point-mass model inherently requires mathematical compromise. For the purpose of this trajectory optimization, the numerical parameters were selected to broadly approximate the performance envelope of a modern Formula 1 vehicle, based on the 2023 F1 technical regulations [3]. While these values do not perfectly replicate a specific chassis or historical generation, they provide a highly realistic baseline for high-downforce, high-power kinematic behavior.

The constants utilized to evaluate the physical constraints within the optimization solver are detailed in Table 1. The idealized vehicle mass of 798 kg approximately aligns with modern premier motorsport regulations. The aerodynamic coefficients (CLA and CDA) dictate the velocity-dependent downforce and drag, while the peak engine power and maximum tractive force establish the longitudinal acceleration boundaries.

Table 1: *Vehicle and Environmental Parameters*

Parameter	Notation	Value
Vehicle Mass	m	798 kg
Peak Engine Power	P_{max}	735.5 kW
Maximum Tractive Force	F_{gear_max}	10 kN
Minimum Turning Radius	R_{min}	5.0 m
Coefficient of Lift $\times$ Area	CLA	4.8 m ²
Coefficient of Drag $\times$ Area	CDA	1.2 m ²
Air Density	ρ	1.225 kg/m ³

3 Problem Definition and Implementation

This section aims to define the mathematical optimization problems which will be analyzed, as well as present the methods used. Firstly, the simple curvature optimization problem will be discussed. Then, the physical lap time optimization will be discussed, as well as the relevant model choices.

3.1 Curvature Optimization

Mathematically, the simple curvature optimization problem is a box-constrained non-linear programming problem with linear equality constraints. Following the standard negative null form, this is defined in Equation (18).

$$\begin{aligned}
& \min_{\mathbf{u}} && f_{curv}(\mathbf{u}) \\
& \text{subject to:} && \mathbf{g}_{lb} \leq \mathbf{u} \leq \mathbf{g}_{ub} \\
& && \mathbf{A}_{eq}(\mathbf{u}) = 0
\end{aligned} \tag{18}$$

Here, the design vector u consists of the N_{ctrl} discrete points representing the lateral deviation from the track centerline, obtained by reducing the full dimension of the circuit coordinates into a smaller set of variables to be handled by the spline interpolation. The design vector is defined in Equation (19).

$$\mathbf{u} = [a_{ctrl,1}, a_{ctrl,2}, \dots, a_{ctrl,N_{ctrl}}]^T \tag{19}$$

Naturally, the objective function f_{curv} for this case follows, with the primary target of minimizing the sum of the curvature. However, during preliminary investigations, a pure curvature minimization results in the optimizer choosing to follow the widest possible lines in all corners (known as the 'outside' line), which naturally minimizes the total

curvature. As such, in order to receive results which are semi-realistic, a small penalty was added to the objective function in the form of a weighted summation of the total arc length. Taking into account the total arc length, the optimizer should avoid drastically increasing the total distance traveled for minimal curvature improvements. The objective function is shown in Equation (20), with the tuning weight w_{length} and the local distance Δs_i .

$$f_{curv}(\mathbf{u}) = \sum_{i=1}^{N_{ctrl}} (\kappa_i^2 \Delta s_i + w_{length} \Delta s_i) \quad (20)$$

Box constraints are imposed upon the optimization in the form of track boundary constraints for each design variable $a_{ctrl,i}$ in the design vector u . These constraints are defined as the distance of a control point $a_{ctrl,i}$ from the left and right boundary (upper and lower) boundary respectively, and are shown in Equation (21). Here, W_l and W_r represent the left and right distance to the bound from the centerline respectively. An arbitrary 'safety margin' is also imposed on the constraints to take into account the physical dimensions in the car, in the form of a car margin parameter $c_m = 0.5$ meters. These constraints were chosen instead of nonlinear constraints as they drastically reduce the computational time and memory requirements for the optimization.

$$\begin{aligned} g_{ub}(\mathbf{u}) &= W_l(\mathbf{u}) - c_m \\ g_{lb}(\mathbf{u}) &= -W_r(\mathbf{u}) + c_m \end{aligned} \quad (21)$$

Due to choice of cubic spline interpolation used, box splines work exceptionally well to manage the boundary constraints. It must be noted that B-Splines were also considered in the development of this optimization model. However, their inability to define curves exactly through the control points would require the implementation of true nonlinear constraints, which were found infeasible due to the dimensionality of this project. Furthermore, linear equality constraints are imposed on the start and end positions to account for the periodicity of the circuit. Mathematically, this enforces C^0 position and C^1 heading continuity at the start/finish line to ensure that the resulting path remains smooth and continuous.

In terms of mathematical topology, the resulting optimization is strictly bounded. The feasible region of the design space forms a closed, bounded geometric, thus suggesting that a global minimum is mathematically guaranteed to exist. Due to the non-linear objective function (as a result of the calculation of curvature), the problem is non-monotonic, further suggesting the existence of a global minimum.

Proving mathematical convexity (or non-convexity) in such a problem is inherently difficult. In Cartesian coordinates, the problem is naturally non-convex, as a straight line cannot be drawn between any two valid points on a given circuit without crossing the boundaries. However, this is largely mitigated by the choice of coordinate space. Instead of using simple Cartesian coordinates, the space is transformed into a curvilinear coordinate system, analogous to the Frenet-Serret coordinate system [4]. This mathematically

converts the non-convex polynomial space into a 'hyper-rectangular' space, which is naturally convex. However, some considerations must be made to the limitations of this assumption. Assuming sector independence, there are certain circuit geometries (like double-apex corners) where multiple minima would result in a similar total curvature. This is partially treated by implementing the distance penalty, which ensures the optimizer follows a path similar to the traditional "out-in-out" racing line.

Numerical noise is introduced to the model in various ways. Primarily, this is introduced due to the reduced discretization of the circuit and the numerical interpretation of the derivatives. The choice of cubic spline interpolation aims to reduce the effect of the reduced dimension by minimizing the 'overshoots' between the distinct control points.

As a result of the 'curse of dimensionality', the resulting model is highly sensitive to the input parameters, namely the distance penalty weight w_{length} and the number of reduced control points N_{ctrl} . As such, a sensitivity analysis cannot be performed on individual design variables. Instead, further discussion on sensitivity will be made coincident to the results in Section 4.

3.2 Physical Lap Time Optimization

The mathematical definition of the full lap time optimization problem is similar to the curvature optimization, being a box-constrained nonlinear programming problem with linear equality constraints. Following the standard negative null form, this is defined in Equation (22). Note that the objective function not only depends on the displacement vector $\mathbf{u}$, but also the vector resulting from the physical calculations $\mathbf{v}$.

$$\begin{aligned} \min_{\mathbf{u}} \quad & f_{phys}(\mathbf{u}, \mathbf{v}) \\ \text{subject to:} \quad & \mathbf{g}_{lb} \leq \mathbf{u} \leq \mathbf{g}_{ub} \\ & \mathbf{A}_{eq}(\mathbf{u}) = 0 \end{aligned} \tag{22}$$

The definitions of the design variables and design vector, as well as the relevant box and equality constraints, remain unchanged to the simple curvature optimization problem. Critically however, the objective function has changed substantially. The problem no longer seeks to minimize the curvature, but attempts to minimize the lap time for a given line. To quantify the effects of the physics model on the objective function, the physics vector $\mathbf{v}$ is defined. In the code, this can be interpreted as the velocity curve resulting for a chosen path. The objective function for the full physical lap time optimization is shown in Equation (23). As the minimization of the total arc length is implicitly considering in the lap time minimization, explicitly adding such a penalty is unnecessary.

$$f_{phys}(\mathbf{u}, \mathbf{v}) = \sum_{i=1}^{N_{ctrl}} \frac{\Delta s_i}{v_i} + w_{smooth} \sum_{j=1}^{N_{ctrl}} (\Delta^2 a_{ctrl,j})^2 \tag{23}$$

The implementation of the discrete physical model in this problem has introduced substantial further non-linearity to the problem. As a result of the multi-pass physical

formulation, including the nonlinear aerodynamic factors as well as the $\min()$ functions to determine the local physical constraints, the design space is highly non-convex and the convexity assumption made for the previous case cannot be carried forward. This is partially mitigated by reducing the small-scale nonlinearities which form as sharp second-order discontinuities (steering jerk) in the form of a smoothness penalty, with an analogous tuning weight w_{smooth} . As the path in the model is interpolated using Modified Akima splines, this mathematically acts as a low-pass filter on the optimization and the design space. This smoothing is also used to minimize the non-natural oscillations which may form as a result of the reduced-order spline interpolation used to represent the circuit.

Naturally, similarly to the previous case this problem is also highly sensitive to the input variables, primarily the number of reduced control points N_{ctrl} and the smoothing penalty proportional to w_{smooth} . The monotonicity and shape of the design space remain similar to the previous case.

3.3 Choice of Optimization Models

This section will provide discussion on the choice of optimization model(s) for each problem, the relevant assumptions, and their consequences.

3.3.1 Brief Discussion on Design Space

Choosing an adequate optimization approach for this model proved to be notoriously difficult due to the dimensionality, non-linearity, and non-convexity of the problem(s). As the physical model imposes inherent discontinuities, using a simple gradient descent method is not guaranteed to converge to a global minimum. However, the high dimensionality of the problems also discourages using only metaheuristic methods. Consider this conceptual sketch of the design space, shown in Figure 2.

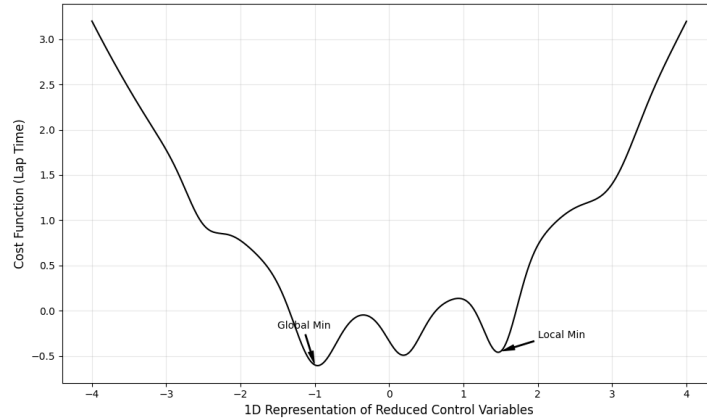


Figure 2: *Conceptual Sketch of Design Space*

Due to the geometric behavior of curvature in such a bounded domain, the design space far away from the minimum behaves like a convex domain. As such, perturbations of the initial guess tend to converge to similar results in the curvature optimization. As a result

of this property of the geometric curvature, similar results follow for the physical case (albeit with a weaker correlation). However, within this 'minimal space', there are many local minima which result from small, local perturbations. These small perturbations are complex and difficult to locate, and are the primary hurdles of implementing gradient-based optimizations methods for these problems. On the other hand, capturing such small perturbations with metaheuristic methods is often impossible at the dimensions required for this problem (hence why these methods often use gradient-based 'smoothing' functions). As no single method can correctly, and efficiently (without brute forcing), capture this design space, a multi-phase optimization must be performed. These methods will primarily use gradient-descent methods to carefully adjust and perturb the design space to seek convergence to a strong, but not necessarily global, minimum.

With the ultimate goal of finding a minimum lap time (and consequently finding an optimal path), the following methodology will be employed. Firstly, a single gradient-descent based algorithm will be implemented for the simple, curvature optimization problem. This choice was made as the design space is sufficiently less complex than the fully physical model and is practically convex. As such, such an optimization will be sufficiently close to the global minimum to be acceptable. The choice was made to use the Sequential Quadratic Programming (SQP) algorithm a Newton method to solve KKT conditions, implemented through MATLAB's *fmincon* function. This algorithm fundamentally subdivides NLP problems into Quadratic Programming (QP) subproblems using Lagrange multipliers and the Lagrangian of the objective function. For each input path, it calculates the gradient and Hessian of the Lagrangian, and uses a second order Taylor series expansion to mimic the local curvature. This local QP problem is then solved and determines the search direction, which is then fed into a standard BFGS algorithm to update the approximation of the Hessian. The implementation of the BFGS algorithm further helps deal with the non-convexity of the design space by mathematically forcing the Hessian to remain positive-definite at each step, improving convergence in near-local minima [5].

Following the curvature optimization, the full physical optimization will be performed similarly using SQP in a multi-phase approach. To quantify the sensitivity of the optimization to the initial guess, two different approaches will be used. The first of these approaches is to provide the curvature-optimized path as the initial guess for the SQP algorithm. As the physical lap time is strongly dependent on the curvature of the chosen path, choosing an initial path with an arbitrarily small curvature is a strong choice to ensure that the algorithm begins in the vicinity of the minimal region. The second approach will attempt to provide a more physically accurate guess by employing the use of the metaheuristic Particle Swarm Optimization (PSO) algorithm at a reduced dimension. This method is used with the rationale that it will be able to provide a non-geometrically optimal line, which may provide adequate perturbations to the initial guess to circumvent certain local minima. The implementation of this method is discussed in the following section.

From the two inputs, the SQP algorithm will locate a minimum within the minimal space of the design space. However, due to the aforementioned properties of this space, further investigation must be made. To search for other nearby minima, the result from the first

physical optimization algorithm will be slightly perturbed by different methods to allow the solver to effectively allow the algorithm to search different locations in this space, with the goal of finding a stronger minimum. This will involve constant, sinusoidal, and stochastic perturbations functions. In summary, the optimization algorithm for the full physical case is as follows:

1. Obtain reduced order space using Modified Akima spline interpolation
2. Compute initial guess using either curvature optimization or PSO
3. Apply perturbation function
4. Compute final optimized lap time and path

3.4 Implementation of Particle Swarm Optimization

Given the highly non-convex nature of the physical racing line optimization problem, a strictly gradient-based solver can very easily start converging to suboptimal local minima when provided with a poor initial guess, for example the track center line. To combat this, a gradient-free approach was initially explored, using a custom implemented Particle Swarm Optimization (PSO) algorithm (app ref).

3.4.1 Mathematical Formulation of PSO

PSO simulates the social behavior of a swarm of particles exploring a multidimensional search space. In the context of this project, the search space dimensionality is defined by the number of spatial control nodes, N_{ctrl} . Therefore, each particle i in a swarm of N_p particles represents a complete candidate racing line, defined by its design vector of lateral deviations, $\mathbf{u}_i \in \mathbb{R}^{N_{ctrl}}$.

At each iteration k , the trajectory of every particle is adjusted by a velocity vector, $\mathbf{v}_i(k)$, which dictates the spatial modification applied to the control nodes. The velocity is updated based on three components: the particle's own inertia, its cognitive memory of its personal best-found trajectory ($\mathbf{p}_{best,i}$), and the social knowledge of the swarm's global best-found trajectory ($\mathbf{g}_{best}$). The velocity update equation is defined as:

$$\mathbf{v}_i(k+1) = w\mathbf{v}_i(k) + c_p r_p (\mathbf{p}_{best,i} - \mathbf{u}_i(k)) + c_g r_g (\mathbf{g}_{best} - \mathbf{u}_i(k)) \quad (24)$$

where w is the inertia weight, c_p and c_g are the cognitive and social acceleration coefficients, and $r_p, r_g \sim U(0, 1)$ are stochastic random matrices that maintain swarm diversity. The particle's new candidate trajectory is then calculated by applying the velocity to its current position:

$$\mathbf{u}_i(k+1) = \mathbf{u}_i(k) + \mathbf{v}_i(k+1) \quad (25)$$

To ensure the swarm generated physically viable racing lines, strict clamping functions were implemented. The position vectors were explicitly bounded at every iteration by the

track limits established in the box constraints ($\mathbf{g}_{lb} \leq \mathbf{u}_i \leq \mathbf{g}_{ub}$), and the velocities were capped at a maximum magnitude $\mathbf{v}_{max} = (\mathbf{g}_{ub} - \mathbf{g}_{lb})$ to prevent divergent oscillations.

3.4.2 Evolutionary Strategy and Curse of Dimensionality

Initially, the custom PSO was deployed as the sole global optimizer for the physical lap-time problem. However, this approach quickly ran into the artifact called "The Curse of Dimensionality". When the track is discretized into 200 spatial control nodes ($N_{ctrl} = 200$), the search space expands to $\mathbb{R}^{200}$. Even if the lateral track width was conceptually discretized into just 20 potential lateral positions per node, the number of possible trajectory combinations scales to 20^{200} . In this huge domain the stochastic nature of the swarm proved to be inefficient. The particles frequently approach highly undesirable local minima, unable to mathematically discover an optimal, continuous racing line through random velocity perturbations.

To mitigate this a hybrid methodology was tested. The PSO was utilized on a coarser control mesh to search for a global valley, outputting a rough initial guess, that was fed into a high-resolution, gradient-based solver (`fmincon`). While this hybrid approach yielded promising results and successfully prevented the gradient solver from stalling on straightaways, it introduced significant computational overhead.

4 Results and Discussion

With the numerical and optimization methodology well established, this section will provide the results and relevant discussion for each case. Firstly, the results of the simple curvature optimization case are discussed. This is followed by discussion on the results of the full physical lap time optimization using the two different initial guess methods. Finally, the results of the local minimum search using perturbation functions will be discussed. Following the discussion of each of the cases, a brief sensitivity analysis will be performed to justify the input parameters used.

Simulations and optimizations were performed on the *Hungaroring*, a circuit located in Budapest, Hungary. This was deemed a good choice, as it contains complex sectors, and is a relatively short track, thus reducing the number of variables which can be used in the reduced discretization. However, it must be noted that this optimization model may be used on any circuit, provided the necessary coordinates of the track are provided. For all gradient-descent cases, iteration tolerances were set to $1e - 6$.

4.1 Results of Curvature Optimization (Simple Problem)

The initial curvature optimization was performed using SQP in the aforementioned method. For this problem, the number of design variables are set to $N_{ctrl} = 350$ and the arc length penalty weight is set to $w_{length} = 0.005$. The value of the penalty weight was arbitrarily chosen as it was shown to provide adequately physical results. The track centerline was provided as the initial guess for the optimization. The primary objective of this approach is strictly geometric: to find a path within the track boundaries that

minimizes the integral of squared curvature, thereby maximizing the effective cornering radii.

Figure 3 illustrates the resulting racing line behavior, with the trajectory color-coded to represent the theoretical velocity profile (the maximum possible velocity which can be achieved with the given car parameters, assuming the path is perfectly adhered to). From a purely kinematic standpoint, the optimizer performs exactly as intended. The generated line exhibits the classic "out-in-out" racing trajectory. It utilizes the full width of the available track on corner entry, sweeps smoothly to approach the geometric apex, and tracks out wide on the exit. By doing so, the optimizer successfully flattens the corners as much as the physical track limits allow, preventing high-frequency steering oscillations and creating a visually continuous, flowing path.

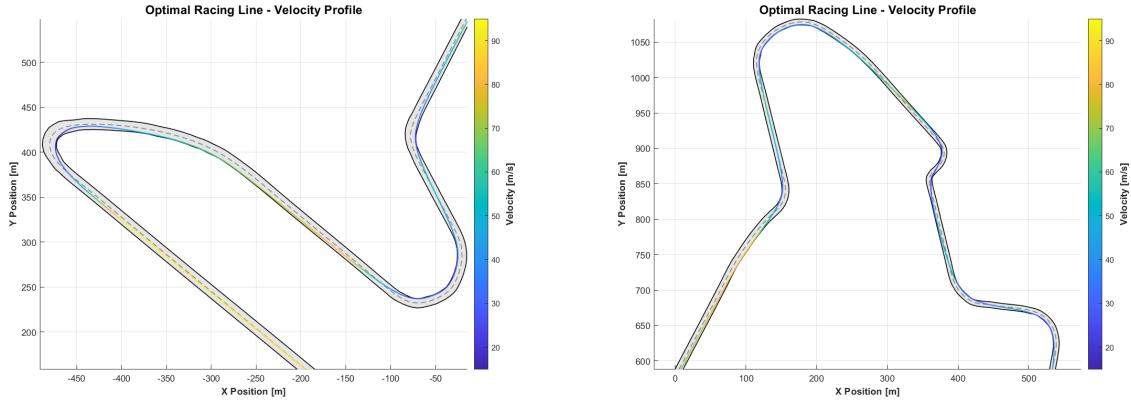


Figure 3: *Resulting Optimal Path Using Simple Curvature Optimization*

However, a critical issue emerges when analyzing the continuity and smoothness of the resulting telemetry data in Figure 4. While the spatial racing line is geometrically smooth, the derived velocity profile is highly erratic. As the curve is the result of a pure geometric optimization, the theoretical velocity is calculated based on the local curvature without sufficient forward-backward smoothing or longitudinal dynamic coupling being considered in its generation. As such, there are many regions with harsh, noisy transitions in speed. Naturally, this is far from optimal. Under these idealized geometric assumptions, the calculation yields a theoretical lap time of 1:29.395 seconds.

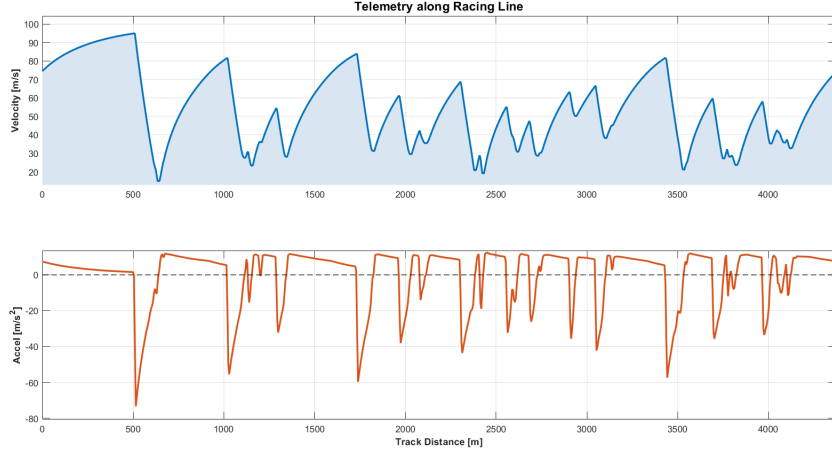
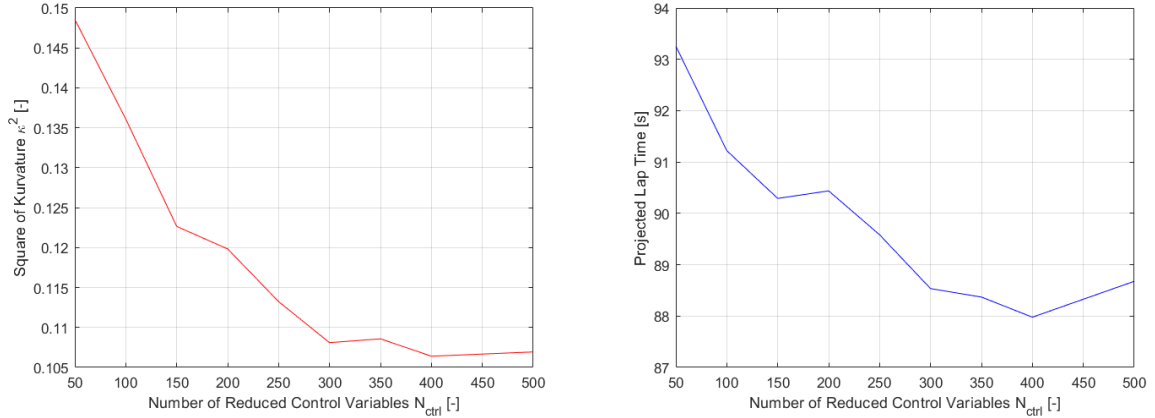


Figure 4: *Telemetry Data for Curvature Optimized Racing Line*

As expected, the curvature optimization thus provides an excellent baseline for the ideal steering path, despite the path itself not being optimized for longitudinal accelerations and true lateral dynamics. This further highlights the simplicity and the effect of the assumptions made in the derivation of this problem, and the necessity of introducing dynamic vehicle constraints or moving toward a full minimum-lap-time optimization.

Briefly, the results of a sensitivity analysis on key input parameter (namely the number of reduced control points N_{ctrl}) will be discussed. The results of this analysis are shown in Figure 5.



(a) *Square of Sum of Arc Curvature vs N_{ctrl}*

(b) *Projected Lap Time vs N_{ctrl}*

Figure 5: *Sensitivity Analysis of Curvature Optimization*

As shown by the two figures, the sum of the curvature generally decreased with an increase in the number of control variables N_{ctrl} . This was directly associated with a reduction in the projected lap time, despite there being no direct implementation of the physics model in the optimization. The increase in the highest number of design

variables was primarily associated to an increase in the oscillations at the highest design variable, a direct consequence of the spline interpolation. Thus, this reaffirms the choice of $N_{ctrl} = 350$ as a balance between discrete accuracy and computation time.

4.2 Full Physics Optimization with Curvature-Optimized Initial Guess

The first approach for the full lap-time minimization uses the previously obtained curvature-optimized racing line as an initial guess to the physical SQP solver. For this problem, the number of design variables $N_{ctrl} = 350$, and the smoothness penalty $w_{smooth} = 0.002$. These values were chosen based off of a sensitivity analysis which is discussed later in this section. Figure 6 illustrates the behavior of the racing line for the same sectors as in the previous section.

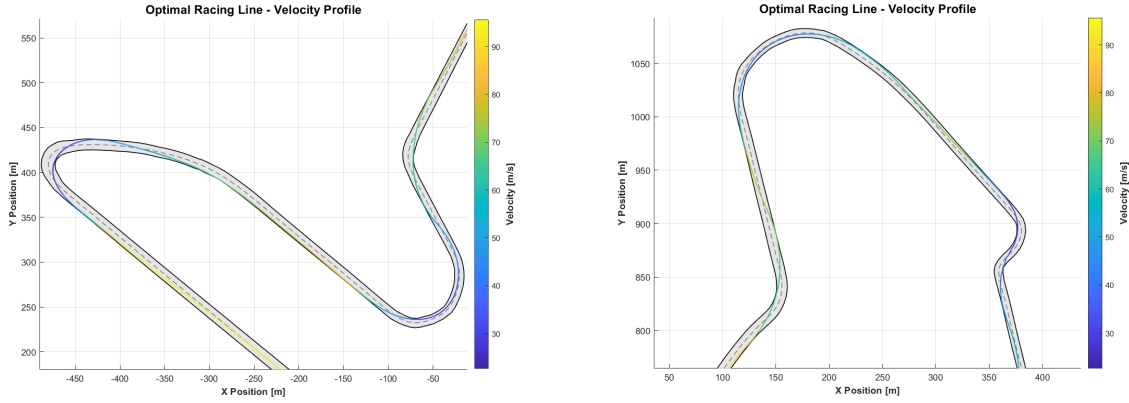


Figure 6: *Resulting Optimal Path Using Physics-Based SQP, Curvature Optimized Input*

Comparing the results of this problem to the curvature optimization, the two methods follow largely similar paths. However, the presence of the physical model has now created a substantially more physical racing line. As the inertia of the vehicle is now accounted for, the optimization must seek a path which maximizes the local achievable velocity in order to take advantage of the aerodynamic capabilities of the vehicle. As such, the near-apex entry and exit trajectories are substantially smoother compared to the curvature optimization. Furthermore, while the curvature optimization resulted in a path which would minimize the time traversing between the inner and outer boundaries at a given corner, this optimization allows for wider lines to be taken at certain points during the lap.

For sharp apex corners such as hairpins as seen in Turn 1 (left of Figure 6) where the velocity must be low, the optimization cannot take advantage of the aerodynamic forces imposed on the vehicle (as downforce scales with velocity squared). As such, the path remains close to the apex to minimize the distance traveled. However, for longer corners, the velocity can be kept substantially higher. As such, the optimizer finds wider lines

which take advantage of the aerodynamic forces more effectively and as such mitigate the effect of the longer distance traveled. The resulting telemetry data for this optimization method is shown in Figure 7.

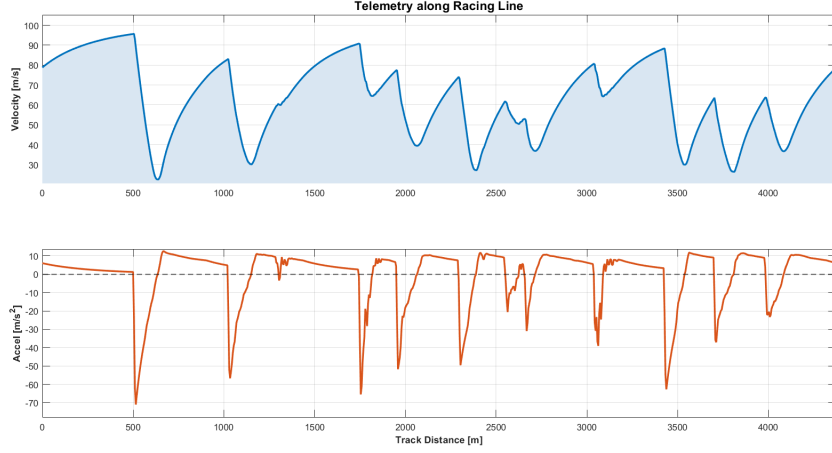


Figure 7: *Telemetry Data for Physics-Based SQP Optimized Racing Line, Curvature Optimized Input*

When compared to the telemetry data from the curvature optimization, substantial differences may be noted. Critically, the velocity graph is significantly smoother, showing that the optimization has yielded a much more physical path. Furthermore, the minimum velocity regions are also substantially higher, particularly in regions where the optimization has exploited the performance gains associated with the aerodynamic forces. Critically, the telemetry data does not show any oscillatory velocity regions. This means that there is no time in which the optimization is not either accelerating or decelerating, thus ensuring that the vehicle’s performance is maximized for the obtained minimum. Thus, this optimization resulted in a lap time of 1:16.264 seconds, substantially faster than the curvature optimized path, and a lap time which is generally representative of those obtained in reality by Formula cars with similar performance parameters. However, due to the simplifications introduced by the physical model, such comparisons cannot be substantive and do not prove the convergence of the model.

A brief discussion will now be made on the sensitivity of this model, the results of which are shown in Figure 8. For this case, the sensitivity analysis will be performed using two input variables, namely the number of reduced control points N_{ctr} and the value of the smoothing parameter w_{smooth} .

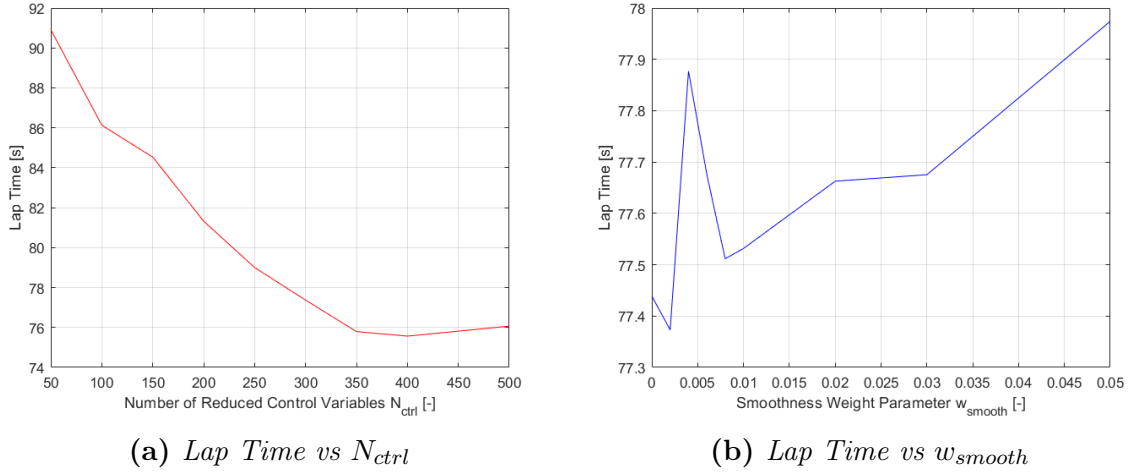


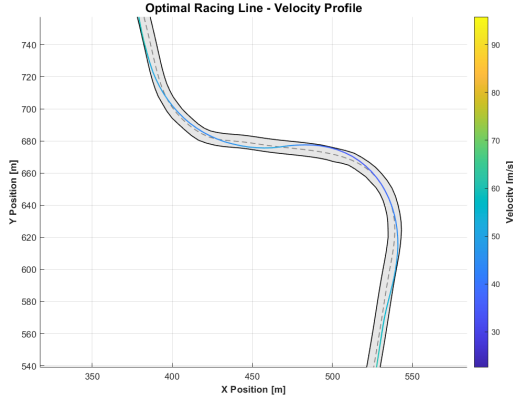
Figure 8: *Sensitivity Analysis of Physics-Based SQP Optimized Racing Line, Curvature Optimized Input*

As shown by the above figures, the obtained lap time is highly sensitive to these two parameters. For the sensitivity to N_{ctrl} , a similar behavior to that of the curvature optimization case is observed, with a similar justification provided. However, the sensitivity to the smoothing parameter is not as trivial. While the general trend shows an increase in lap time with an increase in the smoothing parameter, there is some substantial oscillation in the lower region. As this is a highly dimensional problem, an exact justification for this is difficult to obtain, perhaps attributed to coupling between the smoothness parameter and the number of design variables (the smoothness sensitivity was performed at $N_{ctrl} = 350$). This may also suggest that the discrepancy seen at high N_{ctrl} may be mitigated by increasing the smoothness parameter. However, such a two-dimensional study is not within this report. The chosen parameters indicate the best results, and were chosen as such for the analysis.

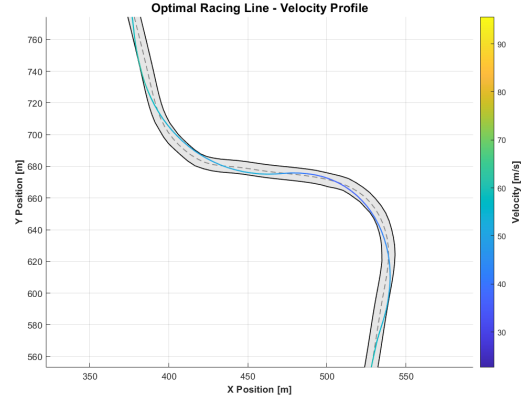
4.3 Full Physics Optimization with PSO Input

As introduced in Section 3.4, a secondary hybrid approach utilizing Particle Swarm Optimization (PSO) to generate the initial guess for the fmincon solver was also evaluated. When analyzing the overarching telemetry data, the speed and acceleration profiles of the PSO-fmincon method are virtually indistinguishable from those produced by the curvature-fmincon approach. However, despite these macroscopic similarities, the PSO-seeded optimization ultimately yielded a lap time approximately one second slower.

A spatial comparison of the generated racing lines reveals the source of this time deficit. While the broader trajectory behavior remains largely consistent between the two methods, the PSO initialization occasionally results in localized, suboptimal cornering phases. Specifically, the PSO-fmincon line occasionally fails to fully utilize the track width, missing the optimal geometric apex. This phenomenon is clearly illustrated in Turn 9 of the Hungaroring, as shown in Figure 9.



(a) *PSO Input - Turn 9 line*



(b) *Curvature Input - Turn 9 line*

Figure 9: *Hungaroring Turn 9 Comparison - PSO vs Curvature Optimization input*

This trajectory artifact is fundamentally tied to the stochastic nature of PSO. As it relies on pseudo-random exploration to position the initial track nodes, it is highly susceptible to prematurely converging on local minima for specific, complex sections of the circuit. If the PSO algorithm settles into one of these localized valleys, it passes a fundamentally flawed initial guess to `fmincon`. Due to `fmincon` being a strictly gradient-based, local optimization algorithm, it cannot step out of that localized attraction. Consequently, it simply polishes the flawed PSO guess, locking the trajectory into a mathematically valid but physically sub-optimal racing line.

These issues ultimately justify the selection of the curvature-based optimization as the primary method. The inherent stochasticity of the PSO algorithm renders its output highly unpredictable for this specific application. Testing demonstrated severe inconsistencies, with resulting lap times fluctuating by approximately ± 5 seconds across multiple runs using identical input parameters but varying random seeds. Furthermore, even the absolute best initial trajectory generated by the PSO was approximately 5 seconds slower than the baseline produced by the purely geometric curvature optimization. Consequently, when this suboptimal guess is passed to the gradient-based solver, `fmincon` is forced to expend significant computational effort merely to recover from the initial time deficit. This heavy reliance on the local optimizer to correct a severely flawed baseline ultimately renders the PSO-hybrid method highly inefficient and unreliable by comparison.

4.4 Final Optimization Using Input Perturbations

The final step of the optimization algorithm is to use a physics optimized path from one of the aforementioned methods, apply small path perturbations, and use it as an initial guess to another final SQP optimization. As a result of the previous analysis, the optimized path from the curvature-optimized initial guess SQP presented in Section 4.2 will be used as an input.

In the search for another minimum, several perturbations of varying type, phase, and

amplitude will be imposed upon the optimized path. These include constant, sinusoidal, and stochastic perturbations. The perturbations were applied to the physical path, and adjustments were imposed to ensure adherence to the boundary positions. The resulting optimized lap times for each perturbation method are shown in Table 2.

Table 2: *Results of Input Perturbations on Optimized Racing Line*

Perturbation Type	Lap Time	Perturbation Type	Lap Time
Unperturbed	1:16.264	Stochastic, $\sigma = 0.1$	1:16.020
Sine, ω	1:16.401	Stochastic, $\sigma = 0.2$	1:16.149
Sine, $\omega/4$	1:15.950	Stochastic, $\sigma = 0.5$	1:17.282
Sine, $\omega/4$, $A = 2$	1:16.536	Constant, +0.5 (Left)	1:15.984
Sine, $\omega/8$	1:16.107	Constant, -0.5 (Right)	1:16.007
Cosine, $\omega/4$	1:16.073		

The resulting lap times shown roughly coherent results. For all three perturbation types, marginal performance improvements were obtained. The lowest lap time was obtained for the sinusoidal forced initial guess (with the angular frequency reduced to one-quarter of its standard value when used as a function of distance), with a final best optimized lap time of **1:15.950**. A constant perturbation of $0.5m$ to the left also improved a close improvement, similar to that obtained from stochastic forcing with the standard deviation $\sigma = 0.1$. The general coherence of the results for small perturbations reaffirm the hypothesized hyper-dimensional shape of the design space, showing that a general minimal space exists with many local minima. By perturbing the result from the initial optimization, multiple improved minima were found which increase the obtained lap time. However, as previously discussed, due to the hurdles this model faces as a result of its nonlinearity and dimensionality, it cannot be proven that the final optimized time represents a true global minimum.

5 Conclusion and Recommendations

This report has successfully outlined the process of racing line and lap time optimization using geometric and discrete point-force dynamics. The numerical implementation was coherently discussed, outlining the discretization of a continuous circuit into N_{ctrl} discrete control points, representing the cartesian discretization in a curvilinear domain, and interpolating the reduced dimension using Modified Akima splines. The derivation of the physical model was also outlined, including the relevant assumptions and their consequences. This was followed by discussion on the optimization problem definitions, the relevant bounds, as well as the topological characteristics of each problem. This allowed for adequate justification into the optimization algorithms and methodology used.

Further, discussion was made on the results obtained for the two optimization cases applied to the Hungaroring. For the simple, geometric curvature optimization problem, the results showed a well-optimized optimal path which was adequately balanced with the length penalty despite the lack of a coherent physical model. The full racing line optimization showed increasingly physical results, with the method of using the curvature-optimized path as an initial guess proving substantially superior at locating the global minimal space (not the global minimum) compared to the case which implemented a Particle Swarm Optimization result as an initial guess. Further analysis on the sensitivities of the methods to the initial parameters was conducted, which found substantial sensitivity to the input parameters. A final SQP-based optimization was conducted using the perturbed output of the curvature-input optimization as an input. Sinusoidal perturbations showed the most significant improvements at locating a strong minimum, with a best optimized lap time of **1:15.950 seconds**.

Notably, this implementation contains several drawbacks. Despite the multi-phase approach making substantial headway at locating a global minimum, it cannot be concluded that the obtained best time is indeed such a minimum. This is a direct result of the high dimensionality of the problem, and perhaps suggests fundamental limitations of such an implementation. Furthermore, the use of point-force dynamics simplifies the true dynamics of a vehicle significantly, and as such the obtained lap time cannot be quantitatively compared to real-life data. The physical model also suffers from a lack of transient dynamics, as well as static friction and aerodynamic parameters.

Further work should primarily investigate ways to mitigate the effects of dimensionality on this problem. This can be performed by the implementation of a sector-by-sector (or turn-by-turn) algorithm, which does not seek to instantaneously optimize the entire track, but performs individual optimizations on each sub-sector of the track. As such, the dimensionality of each individual problem could be substantially reduced, and could perhaps encourage the use of metaheuristic methods. With such improvements on the algorithm, further improvements may be made to the physical model by implementing transient effects, dynamic surface and tire models, three-dimensional inertia dynamics, and other physical factors to improve coherence to reality.

References

- [1] Institute of Automotive Technology (TUM). Racetrack database: A collection of racing tracks for vehicle dynamics simulation. <https://github.com/TUMFTM/racetrack-database>, 2020.
- [2] Hiroshi Akima. A method of univariate interpolation that has the accuracy of a third-degree polynomial. *ACM Transactions on Mathematical Software (TOMS)*, 17(3):341–366, 1991. doi: 10.1145/114697.116810.
- [3] Fédération Internationale de l’Automobile (FIA). 2023 formula 1 technical regulations. Technical Regulation Issue 7, FIA, Paris, France, 2023. URL <https://www.fia.com/regulation/category/110>. Published on 31 August 2023.
- [4] Nicholas M. Patrikalakis, Takashi Maekawa, and Wonjoon Cho. *Shape Modeling*, chapter 2.4 Frenet-Serret formulae. Massachusetts Institute of Technology, December 2009. URL <https://web.mit.edu/hyperbook/Patrikalakis-Maekawa-Cho/node25.html>. Accessed: 17.04.2026.
- [5] Panos Y. Papalambros and Douglass J. Wilde. *Principles of Optimal Design: Modeling and Computation*. Cambridge University Press, Cambridge, 3rd edition, 2017. ISBN 978-1107132672. doi: 10.1017/9781316451038.

A Statement of Fair Contribution

Topic	Involvement Barnabas Szeibert	Involvement Panagiotis Sachinis
Intorduction and Motivation	Checked the section	Wrote this section
Track representation	Checked the code, wrote section	Wrote the code
Adaptive Node Placement	Wrote the code, wrote section	Checked results
Trajectory Interpolation	Together	Together
Physical Model	Together	Together
Curvature Optimization	Checked the code, wrote section	Wrote the code
Particle Swarm Optimization	Wrote the code	Checked the code, wrote section
Physical Lap Time Optimization	Together	Together
Choice of Optimization Models	Checked the section	Wrote this section
Results and Discussion	Together	Together
Conclusion and recommendations	Checked the section	Wrote this section

B Code Implementation

The code created to implement this optimization may be founded in the compressed file submitted alongside this report. The code should contain all necessary modules for it to run. If there are any issues, please do not hesitate to contact the authors.